Git, GitHub, and Gemini CLI Workshop

Overview

AI coding agents are amazing. They can help you move faster, debug issues, and handle repetitive coding tasks with much less effort.

To use AI coding agents safely and effectively on a group project, you need **version control**. In practice, that means using **Git** so everyone can work in parallel, review changes, and avoid breaking shared work.

Git is a bit confusing at first, but AI makes it much easier to use. If you stick to a clear workflow and ask the AI for help when you get stuck, you can be productive quickly without becoming a Git expert on day one.

Intro to Git

Think of Git as a high-tech "Undo" button for your projects. It is the industry standard for version control, ensuring you never lose your work. Git has...

- **Version Tracking**: It keeps a complete history of your project so you can jump back to any previous state if something breaks.
- **Safe Snapshots**: Instead of saving multiple file copies, you take "commits" that capture your entire project at a specific moment.
- **Branching**: You can create "side paths" to test new ideas safely without affecting your main, working version.
- **Team Syncing**: It allows multiple people to work on the same files at once and merges their changes together automatically.

GitHub is a cloud-based platform for hosting git repositories.

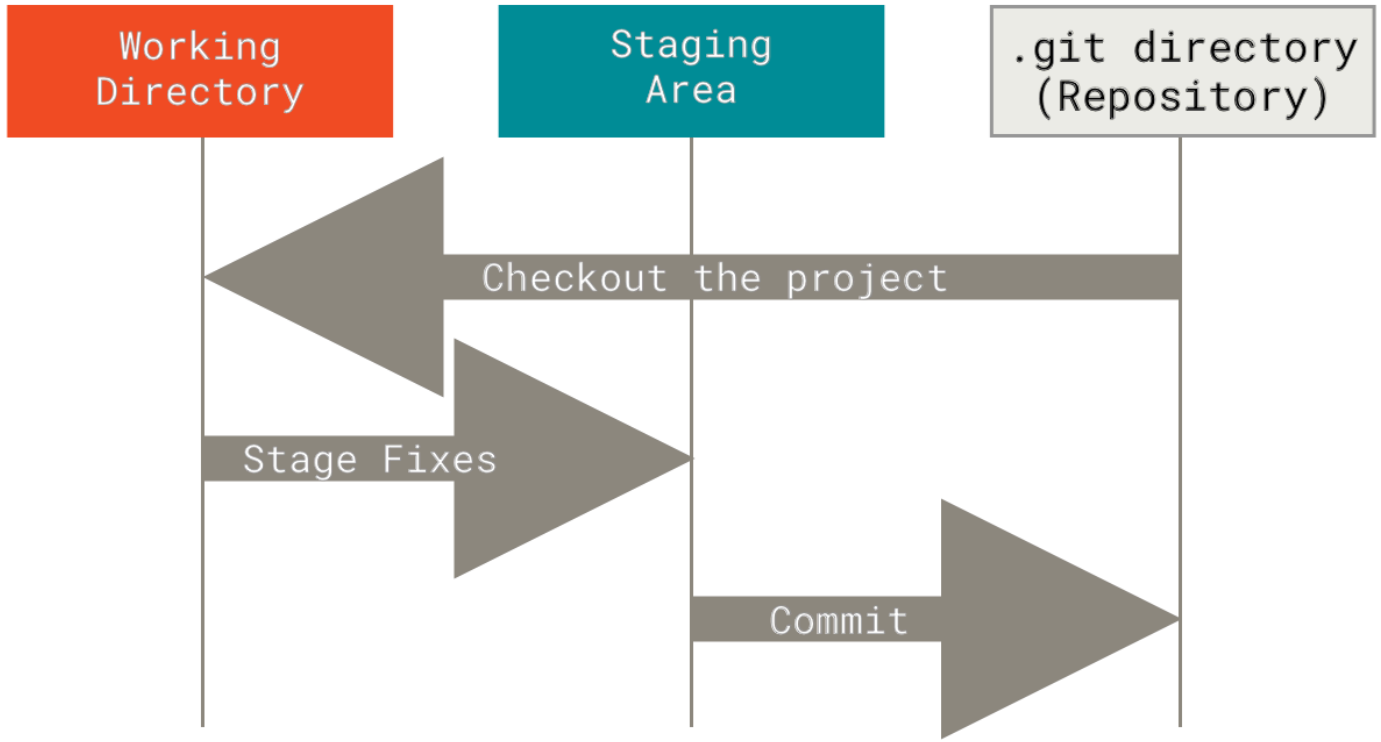
Git Basics, Part 1

Git takes some time to get used to. The best way to learn git is by doing, but it is useful to first understand some basic concepts:

- **Repository:** The folder with all of your files (and the complete history of all the changes you have made). The `.git` sub-folder contains all data required for git to do its thing.
- **Commit:** A snapshot of your repository at a specific point in time.
- **Branch:** Kind of like a "parallel universe" for your code where you can test out new ideas.

Git Basics, Part 2

If you're not used to working with version control, you probably think of the contents of a folder as the one and only truth. With git, what you see in Finder or Explorer is called the **working directory** and is just the version of files that you are currently working on.



The Git Feature Branch Workflow

There are a lot of different ways to use git to collaborate. For our work, we recommend the **feature branch workflow**.

- You **never** work directly on the `main` branch.
- New work happens on a short-lived **feature branch** (e.g., `test_name`). Once you are confident in your changes, you ask the project lead to review and merge into `main`.
- The basic cycle is:

  branch → code → commit → push → pull request → merge

Why this matters: it lets people work in parallel, makes review easier, and lowers the risk of breaking shared code.

If You Get Lost, Ask One of the AIs

Git can be very confusing but you don't need to be an expert to get started and use it effectively. Stick to the recommended workflow and if something goes wrong ask an AI.

Step 0: Workshop Pre-work

If you haven't already done so, please complete the following tasks. For more details, refer to the README one-time setup.

- Install **Git** and **GitHub CLI** (`gh`) on your machine.
- Create/configure your **GitHub account** (including 2FA), get added to the org, and run `gh auth login` .
- Configure **Git identity** (`user.name` and `user.email`), install **Gemini CLI**, and create a local `code` folder for repositories.
- (Optional but recommended) Install **VS Code**

Step 1: Terminal Crash Course

A terminal lets you interact with your computer by typing commands instead of clicking.

Command	What it does
`pwd`	Print working directory — shows where you are
`ls`	List files and folders in the current directory
`cd <folder>`	Change directory — move into a folder
`cd ..`	Move up one level
`cd ~`	Go to your home directory

Try it now: Open Terminal (Mac) or Git Bash (Windows), then run `pwd`, `ls`, and `cd Documents/code`.

Step 2: Clone the Repo

Go to Your Code Folder First:

```
cd ~/Documents/code
pwd
```

Clone the Workshop Repository:

```
git clone https://github.com/dougj892/workshop_test_repo
```

> **VS Code alternative:** Command Palette (`Cmd+Shift+P` / `Ctrl+Shift+P`) → "Git: Clone" → paste the URL → choose your `Documents/code` folder.

cd into the Repository Folder:

```
cd workshop_test_repo
pwd
```

Step 3: Build and Ship a Small Feature

1. Create and Switch to a New Branch:

```
git switch -c test_name
```

Replace `name` with your name.

> **VS Code alternative:** Click the branch name in the bottom-left corner → "Create new branch..." → type `test_name`.

2. Make a Change:

Add some content to the top of `README.md`. It doesn't matter what you add, but you should add it to the top of the file. (This will ensure that this change conflicts with other participants' changes.)

Step 3 (continued)

3. Stage and Commit the Change:

```
git add .
git commit -m "feat: Add new greeting message"
```

`git add .` stages your file changes. `git commit` saves that staged snapshot to your local Git history with a message.

> **VS Code alternative:** In the Source Control panel (branch icon in left sidebar), click **+** to stage files, type a message, and click **Commit**.

Step 3 (continued)

4. Push the Branch to GitHub:

```
git push -u origin test_name
```

This uploads your branch to GitHub. After this, you can just use `git push`.

> **VS Code alternative:** Click **"Publish Branch"** in the Source Control panel. For subsequent pushes, click **"Sync Changes"**.

5. Create a Pull Request:

Navigate to the GitHub repository in your browser. GitHub will usually prompt you to create a pull request from your newly pushed branch. Fill in the details and create the PR.

Step 3 (continued)

6. Review and Merge (Doug to do this):

I will review the PR, potentially suggest changes, and then approve and merge it into the `main` branch.

Step 4: Handling a Merge Conflict

When we tried to merge in the second pull request we got an error saying that the branch has conflicts that must be resolved. This is because the second participant was updating the same section of `README.md` as the first participant.

1. Switch to Main and Pull Latest Changes:

```
git switch main
git pull origin main
```

> **VS Code alternative:** Click the branch name in the bottom-left → select "main". Then click **Sync Changes** (circular arrows in the status bar).

Step 4 (continued)

2. Switch Back to Feature Branch:

```
git switch test_name
```

> **VS Code alternative:** Click the branch name in the bottom-left → select your feature branch.

Step 4 (continued)

3. Merge `main` into Your Feature Branch:

```
git merge main
```

> **VS Code alternative:** Command Palette → "Git: Merge Branch..." → select "main". VS Code highlights conflicts with colored blocks and buttons: **Accept Current Change**, **Accept Incoming Change**, **Accept Both Changes**.

If you get a conflict on the command line:

- Run `git status` to see which files are conflicted
- Open each conflicted file and find the conflict markers (`<<<<<<<` , `=======` , `>>>>>>>`)
- Decide what final text to keep, delete all marker lines, and save

Step 4 (continued)

4. Stage and Commit the Resolved Conflicts:

```
git add .
git commit -m "Merge main into feature branch and resolve conflicts"
```

> **VS Code alternative:** In the Source Control panel, click **+** to stage resolved files, type a message, and click **Commit**.

5. Push: `git push`

> **VS Code alternative:** Click **Sync Changes** in the status bar.

What Are AI Coding Agents?

AI coding agents are tools that can read your codebase, run commands, propose edits, and help you complete software tasks faster.

Agent	Cost	Quality
Codex	Medium	High
Claude Code	Medium-High	High
Gemini CLI	Low-Medium	Medium-High

State of the market as of Feb 2026

Important Safety Warning

AI coding agents can do amazing things quickly, but unsupervised use can also cause real damage, including deleting files, breaking pipelines, leaking secrets, or introducing subtle bugs.

In the longer term, we will likely build stronger safeguards and defaults for users. For now, please **watch what the coding agents are doing** and **don't auto-approve everything!**

Tips for Using AI Coding Agents Well

- Grant the **minimum permissions** needed; avoid broad filesystem and shell access unless required.
- **Never expose secrets** (API keys, credentials, private data) in prompts or unprotected files.
- Ask the agent to **explain planned changes** before major edits.
- **Review every file diff** yourself before accepting changes.
- **Run tests** and key scripts locally after agent edits.
- **Make commits yourself** with clear messages so you control what gets recorded.
- Use **small, focused tasks** instead of one giant prompt.
- If something looks risky or unclear, **stop and ask** for a safer alternative.

AI Coding Agent Test Run

Let's test this out! We often use the Stata ado file `ipacheckcorrections` to make corrections to survey data. One issue is that we accidentally insert leading or trailing spaces in our Excel config file, which leads to an error. Let's fix this using an AI coding agent!

1. Clone IPA's High-frequency checks repo:

```
cd ~/Documents/code
git clone https://github.com/PovertyAction/high-frequency-checks/tree/master
```

AI Coding Agent Test Run (continued)

2. Open Gemini CLI (or another AI coding agent) in the folder:

```
cd ~/Documents/code/high-frequency-checks
gemini
```

3. Let the AI coding agent do its magic

Just use plain language to describe the issue and what you want the coding agent to do. It helps if you tell it that the code you want to change is in the file `ipacheckcorrections.ado`.

If you want to test out the changes, open the ado file and click "do", then call `ipacheckcorrections` from another Stata do file.